WEEK 1: JavaScript Foundations & Node.js Basics

Your First Step to Becoming a Node.js Developer

Welcome to Week 1! This is where everything begins. I know you're probably excited to jump straight into building APIs and working with databases, but trust me - spending this week getting your fundamentals right will make everything else so much easier.

Reality Check: Most interviews in India (whether it's TCS, Infosys, or startups in Bangalore and Pune) start with JavaScript basics. You can't skip this even if you think you know it.

Week 1 Overview

Time Commitment: 2-3 hours daily
What You'll Build: A simple HTTP server that handles different routes
Topics Covered: Modern JavaScript, Async programming, Node.js fundamentals

Why This Week Matters:
Every Node.js concept builds on JavaScript. If you're shaky on promises or don't understand async/await, you'll struggle later. Companies like Flipkart, Zomato, and Swiggy ask these questions in the first round itself.

DAY 1-2: Modern JavaScript Essentials

What You Need to Know

JavaScript has evolved a lot. The ES6+ features (ES2015 onwards) are now standard, and ES2024/ES2025 brought even more improvements. Let's focus on what actually shows up in interviews.

1. let, const, and var - Understanding the Difference

Most Indian companies still ask this basic question to filter candidates.

```javascript
// var - old way (avoid using)
var name = "Rahul";
var name = "Priya"; // Can redeclare - this is a problem!
console.log(name); // Priya

// let - new way for variables that change
let age = 25;
age = 26; // Can reassign
// let age = 27; // Error! Cannot redeclare
console.log(age); // 26

// const - for values that don't change
const PI = 3.14159;
// PI = 3.14; // Error! Cannot reassign
console.log(PI); // 3.14159

// Important: const with objects
const person = {
  name: "Amit",
  city: "Mumbai"
};

// This works! (changing property)
person.city = "Delhi";
console.log(person); // { name: 'Amit', city: 'Delhi' }

// This doesn't work! (reassigning whole object)
// person = { name: "Neha" }; // Error!
```

Interview Question They'll Ask:
"What's the difference between let, const, and var?"

Your Answer Should Be:

- var is function-scoped, let and const are block-scoped
- var can be redeclared, let and const cannot
- const cannot be reassigned (but object properties can change)
- Always prefer const, use let when you need to reassign, avoid var

---

2. Arrow Functions - The Modern Way

javascript

```javascript
// Old function syntax
function add(a, b) {
  return a + b;
}

// Arrow function - shorter
const add = (a, b) => {
  return a + b;
};

// Even shorter (implicit return)
const add = (a, b) => a + b;

// With single parameter (no parentheses needed)
const square = x => x * x;

// With no parameters
const greeting = () => "Namaste!";

console.log(add(5, 3)); // 8
console.log(square(4)); // 16
console.log(greeting()); // Namaste!
```

Real Example - Array Operations:

javascript

```javascript
const numbers = [1, 2, 3, 4, 5];

// Old way
const doubled = numbers.map(function(num) {
  return num * 2;
});

// Modern way (cleaner!)
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6, 8, 10]

// Filtering even numbers
const evens = numbers.filter(num => num % 2 === 0);
console.log(evens); // [2, 4]
```

Important 'this' Behavior:

```javascript
// Regular function - 'this' refers to caller
const person = {
  name: "Suresh",
  regularFunction: function() {
    console.log("Regular:", this.name);
  },
  arrowFunction: () => {
    console.log("Arrow:", this.name);
  }
};

person.regularFunction(); // Regular: Suresh
person.arrowFunction(); // Arrow: undefined (arrow functions don't have their own 'this')
```

---

3. Destructuring - Unpacking Values

This is huge. Every modern codebase uses this.

javascript

```javascript
// Object Destructuring
const student = {
  name: "Priya",
  age: 22,
  city: "Bangalore",
  college: "IIT"
};

// Old way
const name = student.name;
const age = student.age;

// New way (destructuring)
const { name, age, city } = student;
console.log(name); // Priya
console.log(age); // 22

// With different variable names
const { name: studentName, city: location } = student;
console.log(studentName); // Priya
console.log(location); // Bangalore

// With default values
const { name, marks = 85 } = student;
console.log(marks); // 85 (default, since marks wasn't in object)

// Array Destructuring
const colors = ["red", "green", "blue", "yellow"];

const [first, second] = colors;
console.log(first); // red
```

```javascript
console.log(second); // green

// Skip elements
const [, , third] = colors;
console.log(third); // blue

// Rest operator (...)
const [primary, ...others] = colors;
console.log(primary); // red
console.log(others); // ["green", "blue", "yellow"]
```

Real Use Case - Function Parameters:

```javascript
// API response from backend
const userResponse = {
  data: {
    user: {
      name: "Rajesh",
      email: "rajesh@gmail.com",
      phone: "9876543210"
    },
    status: "success"
  }
};

// Extracting nested values
function displayUser({ data: { user: { name, email } } }) {
  console.log(`Name: ${name}, Email: ${email}`);
}

displayUser(userResponse);
// Output: Name: Rajesh, Email: rajesh@gmail.com
```

---

4. Spread and Rest Operators

javascript

```javascript
// Spread operator (...) - expanding arrays/objects

// Combining arrays
const indianCities = ["Mumbai", "Delhi", "Bangalore"];
const moreCities = ["Chennai", "Kolkata"];
const allCities = [...indianCities, ...moreCities];
console.log(allCities);
// ["Mumbai", "Delhi", "Bangalore", "Chennai", "Kolkata"]

// Copying arrays (important!)
const original = [1, 2, 3];
const copy = [...original];
copy.push(4);
console.log(original); // [1, 2, 3] (unchanged)
console.log(copy); // [1, 2, 3, 4]

// Spreading objects
const basicInfo = {
  name: "Amit",
  age: 25
};

const completeInfo = {
  ...basicInfo,
  city: "Pune",
  job: "Developer"
};
console.log(completeInfo);
// { name: "Amit", age: 25, city: "Pune", job: "Developer" }
```

```javascript
// Rest operator - collecting remaining items
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}

console.log(sum(1, 2, 3)); // 6
console.log(sum(10, 20, 30, 40)); // 100

// Real example - function with multiple params
function createUser(name, email, ...otherDetails) {
  return {
    name,
    email,
    details: otherDetails
  };
}

const user = createUser("Neha", "neha@example.com",
"9876543210", "Mumbai", "Engineer");
console.log(user);
// {
//   name: "Neha",
//   email: "neha@example.com",
//   details: ["9876543210", "Mumbai", "Engineer"]
// }
```

---

5. Template Literals - Better String Formatting

javascript

```javascript
// Old way (painful)
const name = "Rohit";
const age = 24;
```

```javascript
const message = "Hello, my name is " + name + " and I am " + age
+ " years old";

// New way (clean!)
const message = `Hello, my name is ${name} and I am ${age} years
old`;
console.log(message);
// Hello, my name is Rohit and I am 24 years old

// Multi-line strings (great for HTML/queries)
const html = `
  <div>
    <h1>${name}</h1>
    <p>Age: ${age}</p>
  </div>
`;

// With expressions
const price = 999;
const discount = 10;
const finalPrice = `Final Price: ₹${price - (price * discount /
100)}`;
console.log(finalPrice); // Final Price: ₹899.1

// Real scenario - Building MongoDB query
const email = "user@example.com";
const query = `db.users.findOne({ email: "${email}" })`;
console.log(query);
```

---

6. Array Methods - Must Know for Interviews

These show up in every coding round. Master them!

javascript

```javascript
const numbers = [1, 2, 3, 4, 5];

// map() - transform each element
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6, 8, 10]

// filter() - keep elements that match condition
const evens = numbers.filter(num => num % 2 === 0);
console.log(evens); // [2, 4]

// reduce() - combine into single value
const sum = numbers.reduce((total, num) => total + num, 0);
console.log(sum); // 15

// find() - get first matching element
const found = numbers.find(num => num > 3);
console.log(found); // 4

// some() - check if any element matches
const hasEven = numbers.some(num => num % 2 === 0);
console.log(hasEven); // true

// every() - check if all elements match
const allPositive = numbers.every(num => num > 0);
console.log(allPositive); // true
```

Real Example - Processing User Data:

```javascript
const users = [
  { name: "Amit", age: 25, city: "Mumbai", salary: 500000 },
  { name: "Priya", age: 28, city: "Bangalore", salary: 800000 },
  { name: "Rahul", age: 22, city: "Delhi", salary: 400000 },
  { name: "Sneha", age: 30, city: "Pune", salary: 1000000 }
];
```

```javascript
// Get names of all users from Bangalore
const bangaloreUsers = users
  .filter(user => user.city === "Bangalore")
  .map(user => user.name);
console.log(bangaloreUsers); // ["Priya"]

// Calculate total salary expense
const totalSalary = users.reduce((total, user) => total +
user.salary, 0);
console.log(totalSalary); // 2700000

// Find high earners (salary > 7 lakhs)
const highEarners = users.filter(user => user.salary > 700000);
console.log(highEarners);
// [{ name: "Priya", ... }, { name: "Sneha", ... }]

// Check if any user is from Chennai
const hasChennaiUser = users.some(user => user.city ===
"Chennai");
console.log(hasChennaiUser); // false
```

Practice Exercise 1 (Day 1-2)

Try these on your own before looking at solutions:

```javascript
// Exercise 1: Create an array of 5 products with name, price,
category
// Filter products under ₹500
// Calculate total price of filtered products

// Exercise 2: Create a function that takes a name and returns
```

```
// "Good Morning, [name]!" using template literals

// Exercise 3: Given this object:
const order = {
  id: 101,
  customer: "Rajesh",
  items: ["Laptop", "Mouse", "Keyboard"],
  total: 55000
};
// Use destructuring to extract customer and total

// Exercise 4: Merge these two arrays and remove duplicates
const arr1 = [1, 2, 3, 4];
const arr2 = [3, 4, 5, 6];
```

DAY 3-4: Asynchronous JavaScript

This is where things get interesting. Understanding async code is CRITICAL for Node.js.

Why Async Matters

JavaScript is single-threaded. If you make a database call or read a file synchronously, your entire application freezes. That's why we need async operations.

Real Scenario:
Imagine your food delivery app (like Swiggy). When someone places an order:

1. Check if restaurant is open (database call)
2. Check if delivery person is available (database call)
3. Process payment (API call to payment gateway)
4. Send SMS to customer (API call)

If these happened synchronously, each step would block the next. With async, they can happen in parallel or non-blocking way.

---

1. Callbacks - The Old Way

javascript

```javascript
// Simple callback example
function greet(name, callback) {
  console.log(`Hello, ${name}!`);
  callback();
}

function afterGreeting() {
  console.log("Greeting complete!");
}

greet("Priya", afterGreeting);
// Output:
// Hello, Priya!
// Greeting complete!

// Real example - Reading file (simulated)
function readFile(filename, callback) {
  console.log(`Reading ${filename}...`);

  // Simulate delay
  setTimeout(() => {
    const data = "File contents here";
    callback(null, data); // null means no error
  }, 2000);
}

readFile("data.txt", (error, data) => {
  if (error) {
    console.log("Error:", error);
  } else {
    console.log("Data:", data);
  }
```

```
});
```

The Problem: Callback Hell (Pyramid of Doom)

javascript

```javascript
// Scenario: User login flow
// 1. Validate credentials
// 2. Fetch user data
// 3. Fetch user orders
// 4. Fetch order details

validateUser(username, password, (err, user) => {
  if (err) {
    console.log("Validation failed");
  } else {
    fetchUserData(user.id, (err, userData) => {
      if (err) {
        console.log("Fetch failed");
      } else {
        fetchOrders(userData.id, (err, orders) => {
          if (err) {
            console.log("Orders fetch failed");
          } else {
            fetchOrderDetails(orders[0].id, (err, details) => {
              if (err) {
                console.log("Details fetch failed");
              } else {
                console.log("Finally got the data!", details);
              }
            });
          }
        });
      }
    });
  }
```

```
});

// This is hard to read and maintain!
```

---

2. Promises - The Better Way

javascript

```
// Creating a promise
const myPromise = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("Operation successful!");
  } else {
    reject("Operation failed!");
  }
});

// Using a promise
myPromise
  .then(result => {
    console.log(result); // Operation successful!
  })
  .catch(error => {
    console.log(error);
  });

// Real example - Simulating API call
function fetchUserFromAPI(userId) {
  return new Promise((resolve, reject) => {
    console.log(`Fetching user ${userId}...`);

    setTimeout(() => {
```

```
      if (userId > 0) {
        resolve({
          id: userId,
          name: "Amit Kumar",
          email: "amit@example.com"
        });
      } else {
        reject("Invalid user ID");
      }
    }, 1500);
  });
}

// Using it
fetchUserFromAPI(101)
  .then(user => {
    console.log("User fetched:", user);
    return user.id; // This gets passed to next .then()
  })
  .then(userId => {
    console.log("User ID:", userId);
  })
  .catch(error => {
    console.log("Error:", error);
  })
  .finally(() => {
    console.log("Fetch attempt complete!");
  });
```

Chaining Promises - Much Cleaner:

```
javascript

// Same login flow, but with promises
function validateUser(username, password) {
  return new Promise((resolve, reject) => {
```

```
    setTimeout(() => {
      if (username && password) {
        resolve({ id: 1, username });
      } else {
        reject("Invalid credentials");
      }
    }, 1000);
  });
}

function fetchUserData(userId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve({ id: userId, name: "Rajesh", city: "Mumbai" });
    }, 1000);
  });
}

function fetchOrders(userId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve([
        { id: 1, item: "Laptop", price: 50000 },
        { id: 2, item: "Mouse", price: 500 }
      ]);
    }, 1000);
  });
}

// Clean promise chain
validateUser("rajesh", "password123")
  .then(user => fetchUserData(user.id))
  .then(userData => fetchOrders(userData.id))
  .then(orders => {
    console.log("Orders:", orders);
```

```
  })
  .catch(error => {
    console.log("Error:", error);
  });

// Much more readable!
```

---

3. Async/Await - The Modern Way

javascript

```
// Converting promise to async/await
async function getUserOrders() {
  try {
    const user = await validateUser("rajesh", "password123");
    console.log("User validated:", user);

    const userData = await fetchUserData(user.id);
    console.log("User data:", userData);

    const orders = await fetchOrders(userData.id);
    console.log("Orders:", orders);

    return orders;
  } catch (error) {
    console.log("Error:", error);
  }
}

// Call the async function
getUserOrders();

// Look how clean this is compared to callback hell!
```

Real Example - Multiple API Calls:

javascript

```javascript
// Scenario: E-commerce product page
async function loadProductPage(productId) {
  try {
    console.log("Loading product page...");

    // Fetch product details
    const product = await fetchProduct(productId);
    console.log("Product:", product.name);

    // Fetch reviews
    const reviews = await fetchReviews(productId);
    console.log(`Reviews: ${reviews.length} reviews found`);

    // Fetch similar products
    const similar = await
fetchSimilarProducts(product.category);
    console.log(`Similar: ${similar.length} products found`);

    return {
      product,
      reviews,
      similar
    };
  } catch (error) {
    console.log("Error loading page:", error);
    throw error;
  }
}

// Helper functions (simulated)
function fetchProduct(id) {
  return new Promise(resolve => {
```

```
    setTimeout(() => {
      resolve({
        id,
        name: "iPhone 15",
        price: 79999,
        category: "smartphones"
      });
    }, 1000);
  });
}

function fetchReviews(productId) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([
        { user: "Amit", rating: 5, comment: "Great phone!" },
        { user: "Priya", rating: 4, comment: "Good value" }
      ]);
    }, 1200);
  });
}

function fetchSimilarProducts(category) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve([
        { name: "Samsung S24", price: 74999 },
        { name: "OnePlus 12", price: 64999 }
      ]);
    }, 800);
  });
}

// Use it
loadProductPage(101)
```

```
  .then(pageData => {
    console.log("Page loaded successfully!");
    console.log(pageData);
  })
  .catch(error => {
    console.log("Failed to load page");
  });
```

---

4. Parallel vs Sequential Execution

javascript

```
// SEQUENTIAL (Slow) - each waits for previous
async function loadSequential() {
  console.time("Sequential");

  const user = await fetchUserData(1);    // Wait 1s
  const orders = await fetchOrders(1);    // Wait 1s
  const reviews = await fetchReviews(1);  // Wait 1s

  console.timeEnd("Sequential");
  // Total: ~3 seconds
  return { user, orders, reviews };
}

// PARALLEL (Fast) - all start together
async function loadParallel() {
  console.time("Parallel");

  // All three start at the same time
  const [user, orders, reviews] = await Promise.all([
    fetchUserData(1),
    fetchOrders(1),
    fetchReviews(1)
```

```
  ]);

  console.timeEnd("Parallel");
  // Total: ~1 second (time of slowest operation)
  return { user, orders, reviews };
}

// Try both
loadSequential(); // Takes 3 seconds
loadParallel();   // Takes 1 second
```

Real Scenario - Dashboard Loading:

javascript

```
async function loadDashboard(userId) {
  try {
    console.log("Loading dashboard...");

    // These can load in parallel - they don't depend on each
other
    const [profile, notifications, stats, recentActivity] =
await Promise.all([
      fetchUserProfile(userId),
      fetchNotifications(userId),
      fetchUserStats(userId),
      fetchRecentActivity(userId)
    ]);

    console.log("Dashboard loaded!");

    return {
      profile,
      notifications,
      stats,
      recentActivity
```

```
    };
  } catch (error) {
    console.log("Dashboard load failed:", error);
  }
}

// This is much faster than loading each one sequentially!
```

Practice Exercise 2 (Day 3-4)

```
javascript

// Exercise 1: Create a function that simulates ordering food
// Steps: Check restaurant availability → Place order → Confirm
order
// Use promises and async/await

// Exercise 2: Create three functions that return promises:
// - fetchUser() - returns user object after 1s
// - fetchPosts() - returns user's posts after 1.5s
// - fetchComments() - returns comments after 2s
// Load all three in parallel and measure time

// Exercise 3: Handle errors properly
// Create a function that sometimes succeeds, sometimes fails
// Use try-catch to handle errors gracefully
```

DAY 5-6: Node.js Core Concepts

Understanding Node.js

Node.js isn't a programming language - it's a runtime environment that lets you run JavaScript outside the browser. Think of it like this: earlier JavaScript could only run in

Chrome, Firefox, etc. Now with Node.js, you can run JavaScript on your laptop, server, or even Raspberry Pi!

Why Companies Like Zomato, Paytm, and Netflix Use Node.js:

- Handles many concurrent connections (thousands of food orders at once!)
- Fast I/O operations (database reads, API calls)
- Same language for frontend and backend (easier for teams)
- Huge npm ecosystem (don't reinvent the wheel)

---

1. Installing Node.js (If You Haven't Already)

`bash`

```
# Check if Node.js is installed
node --version
# Should show something like: v24.0.0

# Check npm version
npm --version
# Should show something like: 10.8.0

# If not installed, download from nodejs.org
# Choose LTS (Long Term Support) version - currently v24
```

For Indian Students:
Download from:

https://nodejs.org/en/download/

Choose "Windows Installer" or "macOS Installer" based on your system.

---

2. Your First Node.js Program

Create a file called `app.js`:

`javascript`

```
// app.js
console.log("Namaste from Node.js!");

const name = "Rohit";
const city = "Mumbai";

console.log(`My name is ${name} and I'm from ${city}`);

// Basic math
const marks = [85, 92, 78, 88, 95];
const total = marks.reduce((sum, mark) => sum + mark, 0);
const average = total / marks.length;

console.log(`Total: ${total}, Average: ${average}`);
```

Run it:

```
bash

node app.js
```

Output:

```
text

Namaste from Node.js!
My name is Rohit and I'm from Mumbai
Total: 438, Average: 87.6
```

That's it! You just ran JavaScript on your computer without a browser!

---

3. Node.js REPL (Read-Eval-Print-Loop)

REPL is like a playground to test JavaScript code quickly.

```
bash
```

```
# Start REPL
node

# Now you're in REPL mode
> 2 + 2
4

> const name = "Priya"
undefined

> name
'Priya'

> console.log("Hello " + name)
Hello Priya
undefined

> .exit  // or press Ctrl+C twice to exit
```

Pro Tip: Use REPL to quickly test syntax or functions during interviews!

---

4. Node.js Modules System

Node.js uses modules to organize code. Think of modules as separate JavaScript files that you can import and use.

Types of Modules:

1. Core Modules - Built into Node.js (fs, http, path, os)
2. Local Modules - Your own files
3. Third-party Modules - Installed via npm (express, mongoose)

A. Core Modules - File System (fs)

```
javascript
```

```javascript
// filesystem.js
const fs = require('fs');

// Writing to a file
fs.writeFileSync('notes.txt', 'This is my first file created
with Node.js!');
console.log('File created!');

// Reading from a file
const data = fs.readFileSync('notes.txt', 'utf8');
console.log('File contents:', data);

// Appending to a file
fs.appendFileSync('notes.txt', '\nThis is a new line!');
console.log('Content appended!');

// Check if file exists
if (fs.existsSync('notes.txt')) {
  console.log('File exists!');
}

// Delete a file
// fs.unlinkSync('notes.txt');
// console.log('File deleted!');
```

Async Version (Better for production):

```javascript
// filesystem-async.js
const fs = require('fs').promises;

async function fileOperations() {
  try {
    // Write file
    await fs.writeFile('async-notes.txt', 'Hello from async!');
```

```
    console.log('File written');

    // Read file
    const data = await fs.readFile('async-notes.txt', 'utf8');
    console.log('File contents:', data);

    // Append to file
    await fs.appendFile('async-notes.txt', '\nNew line added!');
    console.log('Content appended');

    // Read again
    const newData = await fs.readFile('async-notes.txt',
'utf8');
    console.log('Updated contents:', newData);
  } catch (error) {
    console.error('Error:', error);
  }
}

fileOperations();
```

B. Core Modules - OS Module

javascript

```
// os-info.js
const os = require('os');

console.log('Operating System Info:');
console.log('----------------------');
console.log('Platform:', os.platform()); // win32, darwin, linux
console.log('Architecture:', os.arch()); // x64, arm
console.log('CPU Cores:', os.cpus().length);
```

```javascript
console.log('Total Memory:', (os.totalmem() / 1024 / 1024 /
1024).toFixed(2) + ' GB');
console.log('Free Memory:', (os.freemem() / 1024 / 1024 /
1024).toFixed(2) + ' GB');
console.log('Home Directory:', os.homedir());
console.log('Hostname:', os.hostname());
console.log('Uptime:', (os.uptime() / 3600).toFixed(2) + '
hours');

// Useful for system monitoring in production
```

C. Core Modules - Path Module

javascript

```javascript
// path-examples.js
const path = require('path');

// Current file path
console.log('Current file:', __filename);
console.log('Current directory:', __dirname);

// Join paths (works across OS)
const filePath = path.join(__dirname, 'data', 'users',
'profile.json');
console.log('Joined path:', filePath);

// Get file extension
console.log('Extension:', path.extname('profile.json')); // .json

// Get filename
console.log('Filename:', path.basename(filePath)); //
profile.json
```

```javascript
// Get directory name
console.log('Directory:', path.dirname(filePath));

// Parse path into object
console.log('Parsed path:', path.parse(filePath));
```

---

5. Creating Your Own Modules

math.js (your custom module):

```
javascript
```

```javascript
// math.js
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
const multiply = (a, b) => a * b;
const divide = (a, b) => {
  if (b === 0) {
    throw new Error('Cannot divide by zero!');
  }
  return a / b;
};

// Export multiple functions
module.exports = {
  add,
  subtract,
  multiply,
  divide
};
```

app.js (using your module):

```
javascript
```

```javascript
// app.js
const math = require('./math'); // ./ means current directory

console.log('10 + 5 =', math.add(10, 5));
console.log('10 - 5 =', math.subtract(10, 5));
console.log('10 × 5 =', math.multiply(10, 5));
console.log('10 ÷ 5 =', math.divide(10, 5));
```

Real-world example - User utilities:

javascript

```javascript
// utils/user.js
const users = [];

const addUser = (name, email) => {
  const user = {
    id: users.length + 1,
    name,
    email,
    createdAt: new Date()
  };
  users.push(user);
  return user;
};

const getUser = (id) => {
  return users.find(user => user.id === id);
};

const getAllUsers = () => {
  return users;
};

const deleteUser = (id) => {
  const index = users.findIndex(user => user.id === id);
```

```
  if (index !== -1) {
    users.splice(index, 1);
    return true;
  }
  return false;
};

module.exports = {
  addUser,
  getUser,
  getAllUsers,
  deleteUser
};
```

Using it:

javascript

```
// app.js
const userUtils = require('./utils/user');

// Add users
userUtils.addUser('Amit Kumar', 'amit@gmail.com');
userUtils.addUser('Priya Sharma', 'priya@gmail.com');
userUtils.addUser('Rahul Verma', 'rahul@gmail.com');

// Get all users
console.log('All Users:', userUtils.getAllUsers());

// Get specific user
console.log('User 2:', userUtils.getUser(2));

// Delete user
userUtils.deleteUser(2);
console.log('After deletion:', userUtils.getAllUsers());
```

---

6. Understanding package.json

This file is the heart of every Node.js project. It contains metadata and dependencies.

bash

```bash
# Create package.json
npm init

# Answer the questions or use default
npm init -y  # Creates with default values
```

package.json structure:

json

```json
{
  "name": "my-node-app",
  "version": "1.0.0",
  "description": "My first Node.js application",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js",
    "test": "echo \"No tests yet\""
  },
  "keywords": ["node", "javascript", "backend"],
  "author": "Rohit Kumar",
  "license": "MIT",
  "dependencies": {
    "express": "^4.18.2"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

Installing packages:

```bash
# Install a package and add to dependencies
npm install express

# Install as dev dependency (only needed during development)
npm install --save-dev nodemon

# Install globally (available everywhere on your system)
npm install -g nodemon

# Install all dependencies from package.json
npm install
```

---

7. Creating Your First HTTP Server

This is what you'll be asked in almost every Node.js interview!

Simple server:

```javascript
// server.js
const http = require('http');

// Create server
const server = http.createServer((req, res) => {
  // req = incoming request
  // res = outgoing response

  console.log('Request received:', req.url);
```

```
  // Set response header
  res.writeHead(200, { 'Content-Type': 'text/plain' });

  // Send response
  res.end('Namaste! This is my first Node.js server!');
});

// Listen on port 3000
const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
  console.log('Press Ctrl+C to stop the server');
});
```

Run it:

```
bash

node server.js
```

Open browser:

http://localhost:3000

Server with routing:

```
javascript

// server-routing.js
const http = require('http');

const server = http.createServer((req, res) => {
  const url = req.url;

  // Home page
  if (url === '/' || url === '/home') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
```

```
    res.end('<h1>Welcome to Home Page</h1><p>Namaste!</p>');
  }

  // About page
  else if (url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>About Us</h1><p>We are learning Node.js!</p>');
  }

  // API endpoint (JSON)
  else if (url === '/api/users') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    const users = [
      { id: 1, name: 'Amit', city: 'Mumbai' },
      { id: 2, name: 'Priya', city: 'Bangalore' },
      { id: 3, name: 'Rahul', city: 'Delhi' }
    ];
    res.end(JSON.stringify(users));
  }

  // 404 - Not found
  else {
    res.writeHead(404, { 'Content-Type': 'text/html' });
    res.end('<h1>404 - Page Not Found</h1>');
  }
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

Test all routes:

- http://localhost:3000/
- → Home page
- http://localhost:3000/about

- → About page
- http://localhost:3000/api/users
- → JSON data
- http://localhost:3000/random
- → 404 error

---

8. Handling Different HTTP Methods

javascript

```javascript
// server-methods.js
const http = require('http');

const server = http.createServer((req, res) => {
  const { method, url } = req;

  console.log(`${method} ${url}`);

  // GET request
  if (method === 'GET' && url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>GET Request Received</h1>');
  }

  // POST request (we'll handle body data)
  else if (method === 'POST' && url === '/api/data') {
    let body = '';

    // Collect data chunks
    req.on('data', chunk => {
      body += chunk.toString();
    });

    // When all data received
    req.on('end', () => {
      console.log('Received data:', body);
```

```
      res.writeHead(200, { 'Content-Type': 'application/json'
});
      res.end(JSON.stringify({
        message: 'Data received successfully',
        received: body
      }));
    });
  }

  else {
    res.writeHead(404);
    res.end('Not Found');
  }
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Test POST request using Postman or curl:

bash

```
curl -X POST http://localhost:3000/api/data \
  -H "Content-Type: application/json" \
  -d '{"name":"Amit","email":"amit@example.com"}'
```

DAY 7: Week 1 Mini Project

Project: Simple Task Manager API

Let's build a complete mini-project combining everything you learned!

Project Structure:

text

```
task-manager/
├── app.js
├── data/
│   └── tasks.json
├── utils/
│   ├── fileHandler.js
│   └── taskManager.js
└── package.json
```

1. Initialize project:

bash

```
mkdir task-manager
cd task-manager
npm init -y
```

2. utils/fileHandler.js:

javascript

```
// utils/fileHandler.js
const fs = require('fs').promises;
const path = require('path');

const DATA_FILE = path.join(__dirname, '..', 'data',
'tasks.json');

// Ensure data directory exists
async function ensureDataDir() {
  const dataDir = path.join(__dirname, '..', 'data');
  try {
    await fs.access(dataDir);
  } catch {
```

```
    await fs.mkdir(dataDir);
  }
}

// Read tasks from file
async function readTasks() {
  try {
    await ensureDataDir();
    const data = await fs.readFile(DATA_FILE, 'utf8');
    return JSON.parse(data);
  } catch (error) {
    // If file doesn't exist, return empty array
    return [];
  }
}

// Write tasks to file
async function writeTasks(tasks) {
  await ensureDataDir();
  await fs.writeFile(DATA_FILE, JSON.stringify(tasks, null, 2));
}

module.exports = {
  readTasks,
  writeTasks
};
```

3. utils/taskManager.js:

```
javascript

// utils/taskManager.js
const { readTasks, writeTasks } = require('./fileHandler');

// Get all tasks
async function getAllTasks() {
```

```
  return await readTasks();
}

// Get task by ID
async function getTaskById(id) {
  const tasks = await readTasks();
  return tasks.find(task => task.id === parseInt(id));
}

// Add new task
async function addTask(title, description) {
  const tasks = await readTasks();

  const newTask = {
    id: tasks.length > 0 ? tasks[tasks.length - 1].id + 1 : 1,
    title,
    description,
    completed: false,
    createdAt: new Date().toISOString()
  };

  tasks.push(newTask);
  await writeTasks(tasks);

  return newTask;
}

// Update task
async function updateTask(id, updates) {
  const tasks = await readTasks();
  const index = tasks.findIndex(task => task.id ===
parseInt(id));

  if (index === -1) {
    return null;
```

```
  }

  tasks[index] = { ...tasks[index], ...updates };
  await writeTasks(tasks);

  return tasks[index];
}

// Delete task
async function deleteTask(id) {
  const tasks = await readTasks();
  const filteredTasks = tasks.filter(task => task.id !==
parseInt(id));

  if (tasks.length === filteredTasks.length) {
    return false; // Task not found
  }

  await writeTasks(filteredTasks);
  return true;
}

module.exports = {
  getAllTasks,
  getTaskById,
  addTask,
  updateTask,
  deleteTask
};
```

4. app.js (Main server):

```
javascript

// app.js
const http = require('http');
```

```
const url = require('url');
const taskManager = require('./utils/taskManager');

const server = http.createServer(async (req, res) => {
  const parsedUrl = url.parse(req.url, true);
  const pathname = parsedUrl.pathname;
  const method = req.method;

  // Set CORS headers
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT,
DELETE');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

  // Handle preflight
  if (method === 'OPTIONS') {
    res.writeHead(200);
    res.end();
    return;
  }

  try {
    // GET /api/tasks - Get all tasks
    if (method === 'GET' && pathname === '/api/tasks') {
      const tasks = await taskManager.getAllTasks();
      res.writeHead(200, { 'Content-Type': 'application/json'
});
      res.end(JSON.stringify(tasks));
    }

    // GET /api/tasks/:id - Get single task
    else if (method === 'GET' &&
pathname.startsWith('/api/tasks/')) {
      const id = pathname.split('/')[3];
      const task = await taskManager.getTaskById(id);
```

```
      if (task) {
        res.writeHead(200, { 'Content-Type': 'application/json'
});
        res.end(JSON.stringify(task));
      } else {
        res.writeHead(404, { 'Content-Type': 'application/json'
});
        res.end(JSON.stringify({ error: 'Task not found' }));
      }
    }

    // POST /api/tasks - Create new task
    else if (method === 'POST' && pathname === '/api/tasks') {
      let body = '';

      req.on('data', chunk => {
        body += chunk.toString();
      });

      req.on('end', async () => {
        const { title, description } = JSON.parse(body);

        if (!title) {
          res.writeHead(400, { 'Content-Type':
'application/json' });
          res.end(JSON.stringify({ error: 'Title is required'
}));
          return;
        }

        const newTask = await taskManager.addTask(title,
description || '');
        res.writeHead(201, { 'Content-Type': 'application/json'
});
```

```
        res.end(JSON.stringify(newTask));
      });
    }

    // PUT /api/tasks/:id - Update task
    else if (method === 'PUT' &&
pathname.startsWith('/api/tasks/')) {
      const id = pathname.split('/')[3];
      let body = '';

      req.on('data', chunk => {
        body += chunk.toString();
      });

      req.on('end', async () => {
        const updates = JSON.parse(body);
        const updatedTask = await taskManager.updateTask(id,
updates);

        if (updatedTask) {
          res.writeHead(200, { 'Content-Type':
'application/json' });
          res.end(JSON.stringify(updatedTask));
        } else {
          res.writeHead(404, { 'Content-Type':
'application/json' });
          res.end(JSON.stringify({ error: 'Task not found' }));
        }
      });
    }

    // DELETE /api/tasks/:id - Delete task
    else if (method === 'DELETE' &&
pathname.startsWith('/api/tasks/')) {
      const id = pathname.split('/')[3];
```

```
      const deleted = await taskManager.deleteTask(id);

      if (deleted) {
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ message: 'Task deleted successfully' }));
      } else {
        res.writeHead(404, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ error: 'Task not found' }));
      }
    }

    // Home route
    else if (method === 'GET' && pathname === '/') {
      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.end('<h1>Task Manager API</h1><p>Use /api/tasks to manage tasks</p>');
    }

    // 404
    else {
      res.writeHead(404, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ error: 'Not found' }));
    }

  } catch (error) {
    console.error('Server error:', error);
    res.writeHead(500, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Internal server error' }));
  }
});
```

```
const PORT = 3000;
server.listen(PORT, () => {
  console.log(`✅ Task Manager API running at
http://localhost:${PORT}`);
  console.log(`📝 API Endpoints:`);
  console.log(`   GET    /api/tasks       - Get all tasks`);
  console.log(`   GET    /api/tasks/:id   - Get single task`);
  console.log(`   POST   /api/tasks       - Create new task`);
  console.log(`   PUT    /api/tasks/:id   - Update task`);
  console.log(`   DELETE /api/tasks/:id   - Delete task`);
});
```

5. Test your API:

bash

```
# Start server
node app.js

# Test with curl or Postman:

# Create task
curl -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Learn Node.js","description":"Complete Week 1"}'

# Get all tasks
curl http://localhost:3000/api/tasks

# Get single task
curl http://localhost:3000/api/tasks/1

# Update task
curl -X PUT http://localhost:3000/api/tasks/1 \
  -H "Content-Type: application/json" \
  -d '{"completed":true}'
```

```
# Delete task
curl -X DELETE http://localhost:3000/api/tasks/1
```

Best Resources for Week 1

📺 YouTube Channels (Hindi + English)

Hindi Tutorials:

1. Code With Harry - Node.js Tutorial in Hindi
    - Link:
    - https://www.youtube.com/watch?v=BLl32FvcdVM
    - Duration: ~2 hours
    - Perfect for beginners, explains in simple Hindi
2. Thapa Technical - Node.js Complete Course in Hindi
    - Link: Search "Thapa Technical Node.js Hindi"
    - Very detailed, covers everything step by step
3. Apna College - Node.js Series
    - Good for college students
    - Explains concepts clearly

English Tutorials:

1. Programming with Mosh - Node.js for Beginners
    - Clear explanations, professional quality
2. Traversy Media - Node.js Crash Course
    - Quick and practical
    - Link:
    - https://www.youtube.com/c/TraversyMedia
3. freeCodeCamp - Node.js Full Course
    - Link:
    - https://www.youtube.com/watch?v=f2EqECiTBL8
    - 7+ hours comprehensive course

📚 Written Resources

1. Official Node.js Documentation
    - https://nodejs.org/docs/
    - Most accurate and up-to-date
2. W3Schools Node.js Tutorial
    - https://www.w3schools.com/nodejs/
    - Good for quick reference
3. MDN Web Docs
    - For JavaScript fundamentals
    - https://developer.mozilla.org/en-US/docs/Web/JavaScript

🛠️ Practice Platforms

1. HackerRank - Node.js Practice
2. LeetCode - For JavaScript/Node.js problems
3. Exercism.io - Free coding exercises

💼 Indian Job Market Context

Top Companies Hiring Node.js Developers in India:

- Bangalore: Flipkart, Swiggy, Ola, PhonePe, Razorpay
- Mumbai: Paytm, Dream11, Grofers, Udaan
- Pune: Persistent, Cybage, Zensar
- Delhi/NCR: Zomato, OYO, MakeMyTrip, PolicyBazaar
- Hyderabad: Amazon, Microsoft, Google

Week 1 Checklist

Before moving to Week 2, make sure you can do these:

✅ JavaScript Fundamentals:

- Explain let, const, var differences
- Write arrow functions comfortably
- Use destructuring in code
- Use array methods (map, filter, reduce)
- Work with template literals

✅ Async Programming:

- Explain what callbacks are

- Create and use Promises
- Write async/await code
- Handle errors with try-catch
- Know when to use parallel vs sequential execution

✅ Node.js Basics:

- Explain what Node.js is
- Use core modules (fs, path, os)
- Create your own modules
- Understand package.json
- Create a basic HTTP server
- Handle different routes

✅ Mini Project:

- Built Task Manager API
- Tested all CRUD operations
- Code is pushed to GitHub

Common Interview Questions for Week 1

Q1: What is Node.js and why use it?
Expected answer: Runtime environment for JavaScript, non-blocking I/O, good for I/O-heavy apps

Q2: What is the difference between synchronous and asynchronous code?
Expected answer: Sync blocks execution, async doesn't block, allows concurrent operations

Q3: Explain the event loop
Expected answer: Mechanism that handles async operations, processes callback queue when call stack is empty

Q4: What are callbacks and what problem do they solve?
Expected answer: Functions passed as arguments, enable async operations, but can lead to callback hell

Q5: How are Promises better than callbacks?
Expected answer: Cleaner syntax, better error handling, chainable, avoid callback hell

---

Week 1 Summary

Congratulations! You've completed Week 1! 🎉

What you learned:

- Modern JavaScript (ES6+) features
- Asynchronous programming (callbacks, promises, async/await)
- Node.js fundamentals and core modules
- Creating HTTP servers
- Building a complete CRUD API

Next Week Preview:
Week 2 is about Express.js - a framework that makes building APIs much easier. You'll learn middleware, routing, error handling, and build a more professional API.

Weekend Exercise:
Before starting Week 2, try building one of these:

1. Weather API that reads data from a file
2. Simple blog API with create, read, update, delete posts
3. Contact management system

Push your code to GitHub and add a README file explaining your project!